

DBCWorkspace: An Internal Training Platform for an ICPC

Team

A Technical Report

Cristobal Longares

Instituto Tecnológico de Zitácuaro (ITZ)

July 2026

Abstract

DBCWorkspace is a web application I built for my university's ICPC (International Collegiate Programming Contest) team to replace the mix of a Notion doc, a spreadsheet, and word of mouth we were using to track training. It is a Next.js 14 application backed by a PostgreSQL database through Prisma, with invitation-only authentication, a custom problem/tag/solution model, a hand-built contest-simulation mode, and a per-contest rating scorecard I designed myself rather than adopting an existing Elo-style formula. This report documents the problem the tool addresses, how the system is put together, what actually works today, and a set of things that did not go smoothly during the roughly two weeks I spent building it, including a Vercel build-cache bug involving a stale Prisma Client and a case-sensitivity issue between my Windows development machine and Vercel's Linux build environment.

1 Problem Statement

Competitive programming teams that train together need a shared, queryable record of what they have done: which problems each member has solved, what the reasoning behind a solution was, how a simulated contest actually went once it was over, and which algorithmic areas the team is weak in. Before this project, my team did not have that. Problem lists lived in a spreadsheet that nobody kept updated consistently, solution write-ups (when they existed at all) were scattered across chat messages, and post-contest reflection — the "what worked, what failed" retrospective that ICPC teams are supposed to do after every contest — essentially never happened because there was no dedicated place to write it down.

None of that data being centralized meant none of it was usable. You could not ask "which of us has the weakest graph theory coverage" or "how did our solve times trend across the last five simulated contests," because the underlying data simply was not captured anywhere structured. The goal of this project was to fix that: put problems, solutions, contests, and training history into one relational database, and build a UI around the actual training loop rather than a generic problem tracker.

This is a tool for one specific team, not a public platform. There is no public problem archive, no open registration, and no attempt to generalize beyond how my team specifically trains.

2 Methodology and Implementation

2.1 Stack

The application is built with Next.js 14 using the App Router, TypeScript throughout, Tailwind CSS for styling, Prisma as the ORM, and PostgreSQL (hosted on Neon) as the database. Authentication runs through NextAuth's Credentials provider with JWT sessions. Table 1 summarizes the main choices and the concrete reason behind each one.

Choice	Reason
Next.js 14, App Router	Server Components let list pages query Prisma directly for reads without a separate API layer; route groups (<code>(auth)</code> , <code>(dashboard)</code>) cleanly separate public pages from authenticated ones.
Prisma + PostgreSQL	The domain is relational by nature — problems have tags, contests have submissions per problem per user, topics link to exercises — so foreign keys and real migrations were a better fit than a document store.
NextAuth (Credentials)	JWT sessions with no OAuth app to register, fitting an invitation-only flow instead of working around it.
Zod	Every service function that mutates data validates its input through a schema in <code>src/lib/validations/</code> before touching Prisma.
react-markdown + remark-math + rehype-katex	Problem statements and solutions are full of math; this gives real LaTeX rendering without the friction of a WYSIWYG rich-text editor.
nuqs	Keeps table filters synced to the URL so a filtered problem list is shareable and survives a refresh.

Table 1: Stack choices and their concrete justification.

2.2 Data Model

The Prisma schema (`prisma/schema.prisma`) groups into six areas: users and invitations, problems/tags/solutions/editorials, contests/submissions/post-mortems/rating snapshots, training calendars and sessions, a topic notebook (`NotebookEntry`), and a `ChangeLog` audit table. Every service that mutates a Problem, Solution, Editorial, Contest, or NotebookEntry writes an entry to `ChangeLog` through a shared `logChange()` helper, which is what feeds the "recent activity" list on the dashboard — it is a real audit trail, not a mocked-up feed.

Authentication is invitation-only by design. There is no public sign-up route. An admin creates an `Invitation` row with a token generated by `crypto.randomBytes(32)` and a seven-day expiry; the invitee sets a name and password at `/invite/[token]`, and the password is hashed with `bcryptjs` before being stored. For a team of five to ten people, this was simpler than registering an OAuth application, and open registration made no sense for what is effectively a private internal tool.

2.3 Rating: A Custom Scorecard, Not Elo

One design decision I want to be explicit about is that the "rating" shown on the dashboard is not an Elo-style rating. I did not want to fake a running skill estimate over time the way Codeforces rating works, since that requires assumptions (like an initial rating and K-factor tuning) I had no real basis for with a five-person team and a handful of simulated contests.

Instead, `rating.service.ts` computes, for every problem a user solved in a contest whose status has been set to `FINISHED`, a percentile based on how fast they solved it relative to teammates who also solved it:

$$\text{percentile} = \begin{cases} 100 & \text{if only one solver} \\ \frac{(n - 1 - \text{rank})}{n - 1} \times 100 & \text{otherwise} \end{cases}$$

where n is the number of teammates who solved that problem and `rank` is the solver's position when sorted by solve time (rank 0 is fastest). That percentile is multiplied by a difficulty weight (`EASY` = 1, `MEDIUM` = 2, `HARD` = 3, `UNRATED` = 1) and reduced by 5 points for every wrong submission before the accepted one:

$$\text{score} = \max(0, \text{percentile} \times \text{weight} - \text{attempts} \times 5)$$

A user's rating for that contest is the average of their per-problem scores, stored as one `RatingSnapshot` row per (`user`, `contest`) pair, and recalculated only when `updateContest()` sets a contest's status to `FINISHED` (see Listing 1). It is a per-contest scorecard, not a cumulative ladder — there is no memory of past ratings feeding into the next calculation.

Listing 1: Core scoring loop from `rating.service.ts`.

```
1 const DIFFICULTY_WEIGHT = { EASY: 1, MEDIUM: 2, HARD: 3, UNRATED: 1 };
2 const ATTEMPT_PENALTY = 5;
3
4 sorted.forEach((solve, rank) => {
5   const percentile = sorted.length === 1
6     ? 100
7     : ((sorted.length - 1 - rank) / (sorted.length - 1)) * 100;
8   const score = Math.max(0, percentile * weight - solve.attempts * ATTEMPT_PENALTY);
9   // accumulated per user, averaged, and upserted as a RatingSnapshot
10 });
```

2.4 Skill Radar

The dashboard shows a hexagonal radar chart across six fixed skill areas (Mathematics, Dynamic Programming, Graphs, Data Structures, Geometry, Number Theory), defined in `src/lib/skill-areas.ts`. Rather than pull in a charting library for a single hexagon chart, `SkillRadarChart.tsx` computes the vertex positions directly with trigonometry and renders raw SVG polygons. Percentages per area come from `getSkillStats()` in `skill.service.ts`, which walks every problem's tags, maps each tag to an area through a manually maintained `TAG_TO_AREA` table, and counts how many of that user's solved problems fall in each area. Tags not present in that lookup table (e.g. `two-pointers`, `merge-sort`) do not count toward any area yet — the mapping is incomplete by construction and gets extended as new tags are added to the problem bank.

2.5 Practice and Contest Simulation Mode

The practice page lets a user define one or more "buckets," each specifying a tag, a difficulty, whether to restrict to unsolved problems, and a count. Submitting the form calls `POST /api/problems/random-batch`, which returns a random sample per bucket from `getRandomProblemsBatch()`. The client then starts a countdown timer and persists the whole session — remaining time, the flattened problem list, which ones have been marked solved — to `localStorage`, entirely on the client side, so refreshing the page does not lose the running simulation.

A "hide tags and shuffle" option is on by default. Displaying problems grouped by the bucket they came from would reveal, before opening a problem, which algorithmic technique it needs — which defeats the purpose of simulating a real contest, where that is exactly the information you do not have going in.

2.6 Editor: Markdown and LaTeX Instead of WYSIWYG

Solutions and editorials are authored as Markdown, with LaTeX math delimited by `$...$` and `$$...$$`. The editor component is a plain two-column split: a `<textarea>` on the left and a live preview on the right rendered through `react-markdown` with `remark-gfm`, `remark-math`, `rehype-katex`, and `rehype-highlight`. This replaced an earlier Tiptap-based rich-text editor; the switch is discussed further in Section 4.

2.7 Topic Notebook

`NotebookEntry` models a hierarchy of topics with a self-referential `parentId`, so subtopics can nest under a parent instead of flooding a single flat list. Topics link to practice problems through a separate `TopicProblem` join table, independent of the tag system the practice mode uses. The notebook is seeded from two textbooks, *Competitive Programmer's Handbook* and *Competitive Programming 4*, through `prisma/seed-books.ts`: 48 topics across 8 categories, each with a definition, a note on why it matters, and a worked C++ example. A second seed script, `prisma/seed-problems.ts`, loads 197 problems across 12 topics, pulled from the real Codeforces API (so difficulty ratings reflect actual Codeforces data) supplemented with CSES and SPOJ problems for data structures. Codeforces does not tag on its own, such as Fenwick trees and sparse tables.

3 Results: How It Works Today

The application currently supports the full loop it was designed for: an admin invites teammates, problems get added with tags and a difficulty, team members write solutions and editorials in Markdown with rendered math, contests get created with a set of problems and ICPC-style submission tracking (verdict, minutes from contest start), and closing a contest triggers the rating recalculation described above. The dashboard aggregates all of this — solved-problem counts, upcoming contests, the skill radar, current ratings, and a live activity feed — into a single view per user.

Problem difficulty is deliberately hidden throughout the interface (list view, detail view, practice mode, simulation mode) so a team member cannot see a problem's rating before attempting it, since that number is itself a spoiler about how hard the problem is expected to be. Difficulty *filters* remain available, since filtering by difficulty is a choice the user makes on purpose, unlike a difficulty badge sitting next to a specific problem they have not solved yet.

The 197-problem practice bank and 48-topic notebook are both currently seeded and live in the database used for development; the schema has gone through five migrations as the data model

grew (adding rating snapshots, the topic–problem link, the topic hierarchy, and a fix to allow user deletion).

4 Conclusions and Lessons Learned

The project works and the team actually uses it, which was the actual bar I was building toward. A few things did not go smoothly, and I think they are worth recording honestly rather than glossing over.

Case sensitivity between Windows and Vercel’s Linux build broke the deploy more than once. My development machine is Windows, where the filesystem is case-insensitive, so an import like `@/components/ui/select` resolves fine locally even if the actual file is `Select.tsx`. Vercel builds on Linux, where that same import fails. I ended up writing two throwaway Node scripts, `fix.js` and `revert.js`, to bulk-rewrite import paths across `src/` when this happened. Both scripts are still sitting in the repository root, unused, because I never went back to delete them after they had served their purpose. In hindsight I should have picked one casing convention for UI component filenames at the start of the project instead of fixing it after the fact with a find-and-replace script.

Vercel’s build cache produced a genuinely confusing failure. Vercel caches `node_modules` between deployments, which meant a cached install could skip Prisma’s client-generation step, leaving a stale `@prisma/client` that crashed page-data collection during `next build` for any route that imported it. The fix was a one-line `postinstall`: `prisma generate` hook in `package.json`, but diagnosing it took longer than the fix itself, since the build error did not point at Prisma directly.

I switched the write-up editor mid-project. Solutions and editorials originally used a Tiptap rich-text editor. I replaced it with the plain Markdown-plus-KaTeX split view because Tiptap’s math extension added friction that was not worth it given how math-heavy these write-ups actually are. The old files, `RichTextEditor.tsx` and `lib/typstap/math-extension.ts`, are left in the repository as empty stubs rather than deleted, because file deletion was blocked by tool permissions in the middle of that refactor and I did not go back to clean it up afterward. `RichContent.tsx` is dead code for the same reason.

A client-side routing bug silently broke table filters. The problems table’s filter component used a URL-state library with shallow client-side routing by default, which updates the browser URL without re-triggering the Server Component that actually reads `searchParams` and queries the database. The visible symptom was that changing a filter looked like it worked — the URL changed — while the underlying list never refetched. The same table also had no pagination originally, so it rendered every row in the database at once. Both were fixed together, but the bug is a good reminder that updating the URL and updating the data are two different guarantees when Server Components are involved.

The rating formula has not been validated against anything. I designed the percentile-times-difficulty-minus-penalty formula because it felt reasonable, not because I checked it against how ICPC or Codeforces actually compute relative performance. There is no test suite comparing its output to known contest outcomes. Before trusting it for anything beyond a rough weekly signal, I would want to run it against a few real past contests and sanity-check whether the weights actually produce sensible orderings.

Overall, the parts of the system that model the team’s actual workflow closely — the problem bank, the Markdown/LaTeX editor, the invitation-only auth, the topic notebook — turned out solid. The parts I had to guess at, most notably the rating formula, are the ones I would revisit first if I kept working on this.